



Formal verification

In the context of hardware and software systems, **formal verification** is the act of proving or disproving the correctness of a system with respect to a certain formal specification or property, using formal methods of mathematics.^[1] Formal verification is a key incentive for formal specification of systems, and is at the core of formal methods. It represents an important dimension of analysis and verification in electronic design automation and is one approach to software verification. The use of formal verification enables the highest Evaluation Assurance Level (EAL7) in the framework of common criteria for computer security certification.^[2]

Formal verification can be helpful in proving the correctness of systems such as: cryptographic protocols, combinational circuits, digital circuits with internal memory, and software expressed as source code in a programming language. Prominent examples of verified software systems include the CompCert verified C compiler and the seL4 high-assurance operating system kernel.

The verification of these systems is done by ensuring the existence of a formal proof of a mathematical model of the system.^[3] Examples of mathematical objects used to model systems are: finite-state machines, labelled transition systems, Horn clauses, Petri nets, vector addition systems, timed automata, hybrid automata, process algebra, formal semantics of programming languages such as operational semantics, denotational semantics, axiomatic semantics and Hoare logic.^[4]

Approaches

Model checking

Model checking involves a systematic and exhaustive exploration of the mathematical model. Such exploration is possible for finite models, but also for some infinite models, where infinite sets of states can be effectively represented finitely by using abstraction or taking advantage of symmetry. Usually, this consists of exploring all states and transitions in the model, by using smart and domain-specific abstraction techniques to consider whole groups of states in a single operation and reduce computing time. Implementation techniques include state space enumeration, symbolic state space enumeration, abstract interpretation, symbolic simulation, abstraction refinement. The properties to be verified are often described in temporal logics, such as linear temporal logic (LTL), Property Specification Language (PSL), SystemVerilog Assertions (SVA),^[5] or computational tree logic (CTL). The great advantage of model checking is that it is often fully automatic; its primary disadvantage is that it does not in general scale to large systems; symbolic models are typically limited to a few hundred bits of state, while explicit state enumeration requires the state space being explored to be relatively small.

Deductive verification

Another approach is deductive verification.^{[6][7]} It consists of generating from the system and its specifications (and possibly other annotations) a collection of mathematical *proof obligations*, the truth of which imply conformance of the system to its specification, and discharging these obligations using either proof assistants (interactive theorem provers) (such as HOL, ACL2, Isabelle, Rocq (previously known as Coq) or PVS), or automatic theorem provers, including in particular satisfiability modulo theories (SMT) solvers. This approach has the disadvantage that it may require the user to understand in detail why the system works correctly, and to convey this information to the verification system, either in the form of a sequence of theorems to be proved or in the form of specifications (invariants, preconditions, postconditions) of system components (e.g. functions or procedures) and perhaps subcomponents (such as loops or data structures).

Application to software

Formal verification of software programs involves proving that a program satisfies a formal specification of its behavior. Subareas of formal verification include deductive verification (see above), abstract interpretation, automated theorem proving, type systems, and lightweight formal methods. A promising type-based verification approach is dependently typed programming, in which the types of functions include (at least part of) those functions' specifications, and type-checking the code establishes its correctness against those specifications. Fully featured dependently typed languages support deductive verification as a special case.

Another complementary approach is program derivation, in which efficient code is produced from functional specifications by a series of correctness-preserving steps. An example of this approach is the Bird–Meertens formalism, and this approach can be seen as another form of program synthesis.

These techniques can be sound, meaning that the verified properties can be logically deduced from the semantics, or unsound, meaning that there is no such guarantee. A sound technique yields a result only once it has covered the entire space of possibilities. An example of an unsound technique is one that covers only a subset of the possibilities, for instance only integers up to a certain number, and give a "good-enough" result. Techniques can also be decidable, meaning that their algorithmic implementations are guaranteed to terminate with an answer, or undecidable, meaning that they may never terminate. By bounding the scope of possibilities, unsound techniques that are decidable might be able to be constructed when no decidable sound techniques are available.

Verification and validation

Verification is one aspect of testing a product's fitness for purpose. Validation is the complementary aspect. Often one refers to the overall checking process as V & V.

- **Validation:** "Are we trying to make the right thing?", i.e., is the product specified to the user's actual needs?
- **Verification:** "Have we made what we were trying to make?", i.e., does the product conform to the specifications?

The verification process consists of static/structural and dynamic/behavioral aspects. E.g., for a software product one can inspect the source code (static) and run against specific test cases (dynamic). Validation usually can be done only dynamically, i.e., the product is tested by putting it through typical and atypical usages ("Does it satisfactorily meet all use cases?").

Automated program repair

Program repair is performed with respect to an oracle, encompassing the desired functionality of the program which is used for validation of the generated fix. A simple example is a test-suite—the input/output pairs specify the functionality of the program. A variety of techniques are employed, most notably using satisfiability modulo theories (SMT) solvers, and genetic programming,^[8] using evolutionary computing to generate and evaluate possible candidates for fixes. The former method is deterministic, while the latter is randomized.

Program repair combines techniques from formal verification and program synthesis. Fault-localization techniques in formal verification are used to compute program points which might be possible bug-locations, which can be targeted by the synthesis modules. Repair systems often focus on a small pre-defined class of bugs in order to reduce the search space. Industrial use is limited owing to the computational cost of existing techniques.

Industry use

The growth in complexity of designs increases the importance of formal verification techniques in the hardware industry.^{[9][10]} At present, formal verification is used by most or all leading hardware companies,^[11] but its use in the software industry is still languishing. This could be attributed to the greater need in the hardware industry, where errors have greater commercial significance. Because of the potential subtle interactions between components, it is increasingly difficult to exercise a realistic set of possibilities by simulation. Important aspects of hardware design are amenable to automated proof methods, making formal verification easier to introduce and more productive.^[12]

As of 2011, several operating systems have been formally verified: NICTA's Secure Embedded L4 microkernel, sold commercially as seL4 by OK Labs;^[13] OSEK/VDX based real-time operating system ORIENTAIS by East China Normal University; Green Hills Software's Integrity operating system; and SYSGO's PikeOS.^{[14][15]} In 2016, a team led by Zhong Shao at Yale developed a formally verified operating system kernel called CertiKOS.^{[16][17]}

As of 2017, formal verification has been applied to the design of large computer networks through a mathematical model of the network,^[18] and as part of a new network technology category, intent-based networking.^[19] Network software vendors that offer formal verification solutions include Cisco^[20] Forward Networks^{[21][22]} and Veriflow Systems.^[23]

The SPARK programming language provides a toolset which enables software development with formal verification and is used in several high-integrity systems.

The CompCert C compiler is a formally verified C compiler implementing the majority of ISO C.^{[24][25]}

See also

- [Automated theorem proving](#)
- [Model checking](#)
- [List of model checking tools](#)
- [Formal equivalence checking](#)
- [Proof checker](#)
- [Property Specification Language](#)
- [Static code analysis](#)
- [Temporal logic in finite-state verification](#)
- [Post-silicon validation](#)
- [Intelligent verification](#)
- [Runtime verification](#)
- [Software verification](#)
- [Hardware verification](#)

References

1. Sanghavi, Alok (May 21, 2010). "What is formal verification?". *EE Times Asia*.
2. "Common Criteria for Information Technology Security Evaluation Part 5: Pre-defined packages of security requirements" (<https://www.commoncriteriaportal.org/files/ccfiles/CC2022PART5R1.pdf>) (PDF). Retrieved April 15, 2025.
3. Sanjit A. Seshia; Natasha Sharygina; Stavros Tripakis (2018). "Chapter 3: Modeling for Verification". In Clarke, Edmund M.; Henzinger, Thomas A.; Veith, Helmut; Bloem, Roderick (eds.). *Handbook of Model Checking* (<https://doi.org/10.1007/978-3-319-10575-8>). Springer. pp. 75–105. doi:10.1007/978-3-319-10575-8 (<https://doi.org/10.1007%2F978-3-319-10575-8>). ISBN 978-3-319-10574-1.
4. Introduction to Formal Verification (http://embedded.eecs.berkeley.edu/research/vis/doc/VisUser/vis_user/node4.html), Berkeley University of California, Retrieved November 6, 2013
5. Cohen, Ben; Venkataramanan, Srinivasan; Kumari, Ajeetha; Piper, Lisa (2015). *SystemVerilog Assertions Handbook* (4th ed.). CreateSpace Independent Publishing Platform. ISBN 978-1518681448.
6. Ahrendt, Wolfgang; Beckert, Bernhard; Bubel, Richard; Hähnle, Reiner; Schmitt, Peter H., eds. (2016). *Deductive Software Verification - The KeY Book: From Theory to Practice* (1st 2016 ed.). Cham: Springer International Publishing : Imprint: Springer. ISBN 978-3-319-49812-6.
7. Pretschner, Alexander; Müller, Peter; Stöckle, Patrick, eds. (2019). "Building Deductive Program Verifiers - Lecture Notes". *Engineering secure and dependable software systems*. Amsterdam, Netherlands: IOS Press. ISBN 978-1-61499-976-8.
8. Le Goues, Claire; Nguyen, ThanhVu; Forrest, Stephanie; Weimer, Westley (January 2012). "GenProg: A Generic Method for Automatic Software Repair" (https://www.proquest.com/openview/f837f11066e9c9552df9e497064d6d80/1?pq-origsite=gscholar&cbl=21418&casa_token=X2fQSRNgLNwAAAAA:CBjvmrcBLZRqV7kjC-MY74s0wF68lhBvLF6rV4DWZkalzo2KdlU-UzuMU38YgSmXGbwEL2NAr4w). *IEEE Transactions on Software Engineering*. **38** (1): 54–72. doi:10.1109/TSE.2011.104 (<https://doi.org/10.1109%2FTSE.2011.104>). S2CID 4111307 (<https://api.semanticscholar.org/CorpusID:4111307>).

9. Harrison, J. (2003). "Formal verification at Intel". *18th Annual IEEE Symposium of Logic in Computer Science, 2003. Proceedings.* pp. 45–54. doi:10.1109/LICS.2003.1210044 (<https://doi.org/10.1109%2FLICS.2003.1210044>). ISBN 978-0-7695-1884-8. S2CID 44585546 (<http://api.semanticscholar.org/CorpusID:44585546>).
10. Formal verification of a real-time hardware design (<http://portal.acm.org/citation.cfm?id=800667>). Portal.acm.org (June 27, 1983). Retrieved on April 30, 2011.
11. "Formal Verification: An Essential Tool for Modern VLSI Design by Erik Seligman, Tom Schubert, and M V Achutha Kirankumar" (<http://formalverificationbook.com>). 2015.
12. "Formal Verification in Industry" (<http://www.cl.cam.ac.uk/~jrh13/slides/types-04sep99/slides1.pdf>) (PDF). Retrieved September 20, 2012.
13. "Abstract Formal Specification of the seL4/ARMv6 API" (<https://web.archive.org/web/20150521171234/https://sel4.systems/Docs/seL4-spec.pdf>) (PDF). Archived from the original (<http://sel4.systems/Docs/seL4-spec.pdf>) (PDF) on May 21, 2015. Retrieved May 19, 2015.
14. Christoph Baumann, Bernhard Beckert, Holger Blasum, and Thorsten Bormer Ingredients of Operating System Correctness? Lessons Learned in the Formal Verification of PikeOS (<http://www-wjp.cs.uni-saarland.de/publikationen/Ba10EW.pdf>) Archived (<https://web.archive.org/web/20110719110932/http://www-wjp.cs.uni-saarland.de/publikationen/Ba10EW.pdf>) July 19, 2011, at the Wayback Machine
15. "Getting it Right" (<http://www.ganssle.com/rants/gettingitright.htm>) by Jack Ganssle
16. Harris, Robin. "Unhackable OS? CertiKOS enables creation of secure system kernels" (<http://www.zdnet.com/article/certikos-a-hacker-proof-os/>). ZDNet. Retrieved June 10, 2019.
17. "CertiKOS: Yale develops world's first hacker-resistant operating system" (<https://www.ibtimes.co.uk/certikos-yale-develops-worlds-first-hacker-resistant-operating-system-1591712>). *International Business Times UK*. November 15, 2016. Retrieved June 10, 2019.
18. Scroxtion, Alex. "For Cisco, intent-based networking heralds future tech demands" (<http://www.computerweekly.com/news/252434028/For-Cisco-intent-based-networking-heralds-future-tech-demands>). Computer Weekly. Retrieved February 12, 2018.
19. Lerner, Andrew. "Intent-based networking" (<https://blogs.gartner.com/andrew-lerner/2017/02/07/intent-based-networking/>). Gartner. Retrieved February 12, 2018.
20. Kerravala, Zeus. "Cisco brings intent based networks to the data center" (<https://www.networkworld.com/article/965142/cisco-brings-intent-based-networks-to-the-data-center.html>). NetworkWorld. Archived (<https://web.archive.org/web/20231211215137/https://www.networkworld.com/article/965142/cisco-brings-intent-based-networks-to-the-data-center.html>) from the original on December 11, 2023. Retrieved February 12, 2018.
21. "Forward Networks: Accelerating and De-risking Network Operations" (<http://www.insightssuccess.com/forward-networks-accelerating-de-risking-network-operations/>). *Insightssuccess Media and Technology Pvt. Ltd.* Insights Success. January 16, 2018. Retrieved February 12, 2018.
22. "Getting Grounded in Intent-based Networking" (https://images.idgesg.net/assets/2018/01/idx_2018_intent-based_networking.pdf) (PDF). NetworkWorld. Retrieved February 12, 2018.
23. "Veriflow Systems" (<https://www.bloomberg.com/research/stocks/private/snapshot.asp?privcapid=274750862>). Bloomberg. Retrieved February 12, 2018.
24. "CompCert - The CompCert C compiler" (<https://compcert.org/compcert-C.html>). *compcert.org*. Retrieved February 22, 2023.
25. Barrière, Aurèle; Blazy, Sandrine; Pichardie, David (January 9, 2023). "Formally Verified Native Code Generation in an Effectful JIT: Turning the CompCert Backend into a Formally Verified JIT Compiler" (<https://doi.org/10.1145%2F3571202>). *Proceedings of the ACM on Programming Languages*. 7 (POPL): 249–277. arXiv:2212.03129 (<https://arxiv.org/abs/2212.03129>). doi:10.1145/3571202 (<https://doi.org/10.1145%2F3571202>). ISSN 2475-1421 (<https://search.worldcat.org/issn/2475-1421>). S2CID 253736486 (<https://api.semanticscholar.org/CorpusID:253736486>).

